

1. Une description de votre implantation des cas ELam, EApp, EData et ECase dans typeCheck et eval, ainsi que de sexp2Exp pour data et case:

1. typeCheck:
 - a. ELam: La fonction ajoute à l'environnement le lambda (la variable, formée par son symbole et son type) pour ensuite lui assigner un autre type au corps de la fonction, seulement si la déclaration est bien formée.
 - b. EApp: Pour vérifier le type d'une application on doit réviser à chaque fois si la fonction est compatible avec les arguments qu'elle utilise, sinon on retourne une erreur. Cela est fait en faisant l'appel récursif de type check pour l'argument et la fonction afin de voir si la fonction est bien supposée de renvoyer le même type que TArrow déclare qui rentre (le type au bout de la flèche, soit t2 dans le code).
 - c. EData:
 - d. ECase:
2. eval:
 - a. ELam: Il n'y a pas grande chose à faire, l'expression est déjà sous la forme où elle peut directement être évaluée
 - b. EApp: Pour évaluer une application d'une fonction on doit voir le type de l'argument f, si f est une expression lambda on doit élargir l'environnement (ajouter à la liste d'environnements l'expression lambda f) et évaluer l'expression lambda avec le nouvel environnement.
Mais si f est une fonction on doit l'appliquer au moment de l'évaluation de ses arguments dans l'environnement courant.
 - c. EData:
 - d. ECase:
3. sexp2Exp:
 - a. data:
 - b. case:

2. Les difficultés rencontrées. Si des tests échouent, expliquez pourquoi et proposez une piste de solution.

Premièrement la quantité de code à découvrir pour une première fois à été assez longue et puis on a eu besoin de faire plusieurs relectures avant de commencer à coder.

À tout ça nous devons ajouter le fait d'écrire un rapport de 3 pages en critiquant, expliquant et analysant ce que notre code fait.

Les premières erreurs étaient causées par faute de lire et d'essayer de comprendre le code qui été déjà écrit. C'était plutôt des preuves/erreurs attendant que mon code fonctionne et sans tester véritablement s'il marchait ou pas.

Ensuite nous avons oublié de compter les implémentations de sexp2Exp quand on était en train de nous diviser nos tâches à faire entre les 2.

Pour pouvoir tester on a besoin de finir de compléter tous les TODO de la fonction typeCheck sinon on obtiendrait une erreur de cas non-exhaustifs.

J'ai eu vraiment du mal à comprendre comment parser le sexp2exp pour data, je ne comprenais pas comment écrire en code haskell une récursion pour construire la liste des NewDataType. Ce qui m'a le plus aidé à été de suivre la logique du sexp2exp du let, définit au dessus de la fonction. Mais j'avais toujours l'erreur de ne pas construire récursivement 2

fonctions, j'oubliais premièrement la fonction auxiliaire pour passer d'une liste de Sexp à une liste de DataConstructors (fonction `sexp2dataCons`), je faisais seulement une fonction qui essayait de passer de la forme `(Symbol, [DataConstructor])` directement à de `DataConstructor` à `NewDataType` (fonction `dataCons2dataTyp`) sans avoir véritablement parsé les constructeurs à des types `Type`.

Pour construire le parseur de les Data je me suis inspiré du code pour parser les Let.

3. Votre expérience générale avec Haskell et avec ce TP

Je pense que mon principal problème pour faire ce TP est devoir comprendre beaucoup d'autre code qui n'est pas le mien et dont je ne sais pas quelle est sa fonction principale. Le thème des évaluateurs me mélange beaucoup, mais par contre je pense qu'ils sont des choses vraiment évidentes.

Le pdf avec les instructions pour nous guider sur comment faire le devoir à vraiment été très utile, surtout la partie expliquant l'ordre dans lequel on devait coder les choses ainsi que le schéma du pipeline.